PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software

Disciplina: Teste de Software

ANÁLISE DE EFICÁCIA DE TESTES COM TESTE DE MUTAÇÃO

Avaliação e melhoria da suíte de testes do projeto *operacoes-mutante* usando StrykerJS

Aluno: Mateus Araujo Santos

Matrícula: 853703

Repositório: github.com/mateusdsoc/operacoes-mutante

Outubro de 2026

1. ANÁLISE INICIAL

Antes de mexer em qualquer teste rodei a suíte original (50 testes, todos passando) e gerei a cobertura com `npm test -- --coverage`. O resultado foi **98,64% de linhas**, 85,41% de statements, 58,82% de branches e 100% de funções. A única linha não executada era a 112, que calcula a mediana de arrays de tamanho par.

Em seguida configurei o Stryker (`@stryker-mutator/core` e `@stryker-mutator/jest-runner` 10.0, arquivo `stryker.conf.json` com o runner do Jest e `coverageAnalysis: perTest`) e rodei `npx stryker run`. Foram gerados 213 mutantes: 154 mortos, 3 por timeout, 44 sobreviventes e 12 sem cobertura, dando um **mutation score de 73,71%**.

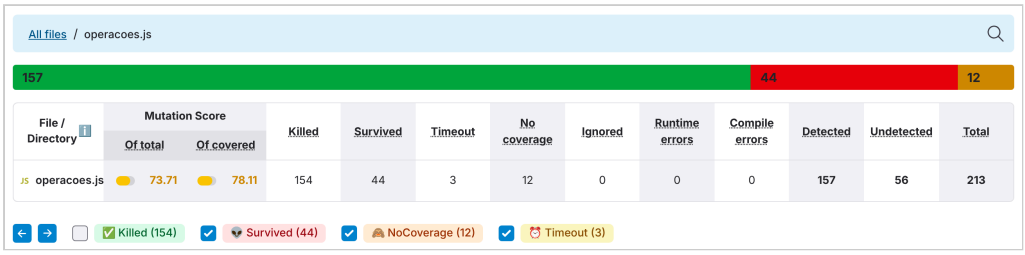


Figura 1 – Resultado da primeira execução do Stryker.

A diferença entre 98,64% e 73,71% mostra que quase toda linha era executada, mas cerca de um em cada quatro bugs introduzidos passava sem nenhum teste falhar. A cobertura só diz que o código rodou; ela não diz se o resultado foi conferido. Olhando os sobreviventes, eles se concentravam em poucos padrões:

- **Só o caso verdadeiro era testado** em `isPar`, `isImpar`, `isDivisivel`, `isMaiorQue`, `isMenorQue`, `isEqual` e `isPrimo`. Trocar a expressão inteira por `true` não quebrava nada.
- **Fronteiras ignoradas:** ninguém testava `raizQuadrada(0)`, `fatorial(0)`, `isPrimo(1)` ou dois números iguais, então trocas como `<` por `<=` sobreviviam.
- **Erros pouco verificados:** a divisão por zero usava `toThrow()` sem mensagem, e os erros de `fatorial`, `inverso`, `raizQuadrada` e dos arrays vazios nem eram testados.
- **Valores de entrada "cômodos":** 0 °C, 32 °F e um array já ordenado, que escondem o efeito da mutação.

2. ANÁLISE DE MUTANTES CRÍTICOS

Mutante 1 – `celsiusParaFahrenheit`, linha 93 (ArithmeticOperator)

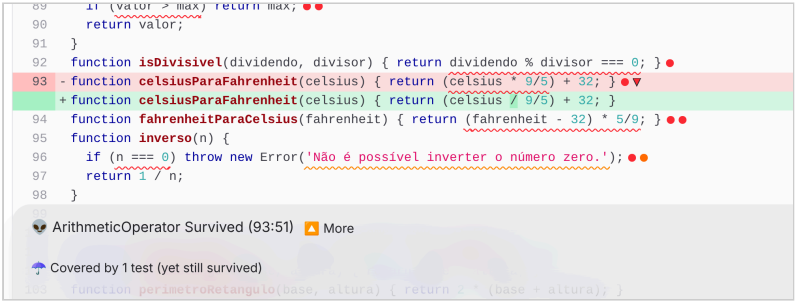


Figura 2 – `celsius * 9/5` trocado por `celsius / 9/5`, sobreviveu.

O único teste era `celsiusParaFahrenheit(0)` esperando 32. Com zero, `0 * 9/5` e `0 / 9/5` dão 0, então o resultado é 32 nas duas versões. O zero anula a multiplicação e o teste acaba verificando só o `+ 32`. A linha estava coberta, mas o valor escolhido tornava a fórmula irrelevante. O mesmo acontecia em `fahrenheitParaCelsius(32)`, porque `32 - 32 = 0`.

Mutante 2 – `medianaArray`, linha 109 (MethodExpression)



Figura 3 – O `.sort(...)` foi removido e o mutante sobreviveu.

O teste original usava `[1, 2, 3, 4, 5]`, que já está ordenado e tem tamanho ímpar. Sem o `sort` o array continua igual e a mediana continua 3. Pelo mesmo motivo sobreviveram as trocas do comparador por `() => undefined` e por `a + b`. Como nenhum teste usava array par, as linhas 111 e 112 também ficaram com mutantes sobreviventes ou sem cobertura.

Mutante 3 – `isMaiorQue`, linha 104 (EqualityOperator)

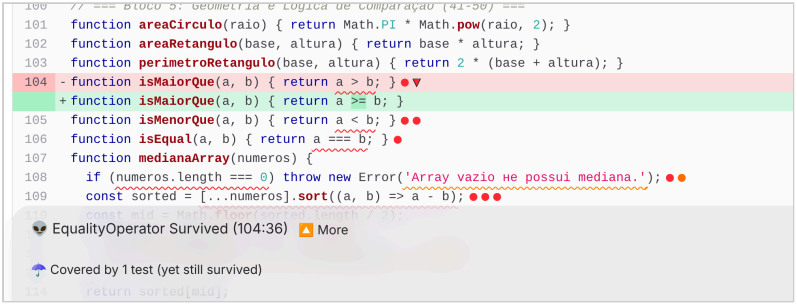


Figura 4 – `a > b` trocado por `a >= b`, sobreviveu.

O único teste era `isMaiorQue(10, 5)`, que dá `true` tanto com `>` quanto com `>=`. Os dois operadores só divergem quando `a === b`, e esse caso nunca foi testado. Como também não havia nenhum teste esperando `false`, o mutante que troca a expressão por `true` sobreviveu. `isMenorQue` tinha exatamente o mesmo problema.

3. SOLUÇÃO IMPLEMENTADA

Adicionei 19 testes (de 50 para 69) no mesmo arquivo `test/operacoes.test.js`, em quatro grupos: erros e mensagens, casos de fronteira, resultados falsos e valores que não escondem o erro. Para os três mutantes acima:

```
expect(celsiusParaFahrenheit(100)).toBe(212);
expect(medianaArray([9, 1, 5])).toBe(5);
expect(medianaArray([8, 2, 6, 4])).toBe(5);
expect(isMaiorQue(5, 5)).toBe(false);
expect(isMaiorQue(3, 8)).toBe(false);
```

- **100 °C → 212 °F:** com `/` o resultado vira ≈ 34,2, e com `* 9 * 5` vira 4532. Qualquer valor diferente de zero já serve. O par `fahrenheitParaCelsius(212) = 100` faz o mesmo na função inversa.
- **[9, 1, 5]:** sem o `sort` o elemento do meio é 1, não 5. O array par `[8, 2, 6, 4]` passa pela linha 112 e mata as trocas de `mid - 1`, do `+` e do `/ 2`, porque cada uma muda o resultado (7, -1 e 20).
- **isMaiorQue(5, 5):** é o ponto em que `>` e `>=` divergem. Já `isMaiorQue(3, 8)` esperando `false` mata o mutante `true`.

Os demais testes seguem a mesma ideia: `toThrow` com a mensagem esperada (mata os *StringLiteral* e os `if (false)`), entradas na fronteira como 0, 1 e arrays vazios, e pelo menos um caso `false` para cada função booleana.

Mutantes equivalentes

Depois dos novos testes sobraram 7 mutantes (96,71%). Analisando cada um, nenhum pode ser morto por um teste razoável, porque o comportamento não muda:

- **`fatorial`, linha 19 (4 mutantes):** o `if (n === 0 || n === 1) return 1` é só um atalho. Sem ele, o `for` começa em 2, não executa e devolve `resultado = 1`. Nenhuma entrada diferencia as versões, então marquei a linha com `// Stryker disable next-line ConditionalExpression,LogicalOperator` e deixei a justificativa no comentário. O Stryker passa a contar 5 mutantes como ignorados: os 4 equivalentes e o `if (true)`, que já estava morto.
- **`produtoArray`, linha 84, e `clamp`, linhas 88 e 89 (3 mutantes):** `[].reduce(fn, 1)` já devolve 1, e no `clamp`, quando o valor é igual ao limite, devolver `min` ou `valor` dá o mesmo número. Cheguei a escrever `clamp(0, 0, 10)` e `clamp(10, 0, 10)` e os mutantes continuaram vivos, o que confirmou isso. Eles só se diferenciariam com entradas fora do propósito da função (`-0` ou algo que não seja array). Por isso preferi deixá-los aparecendo no relatório em vez de escondê-los.

4. RESULTADOS FINAIS

Métrica	Inicial	Final
Testes	50	69
Cobertura de linhas / branches	98,64% / 58,82%	100% / 100%
Mutantes detectados (killed + timeout)	157	205
Sobreviventes / sem cobertura	44 / 12	3 / 0
Ignorados (equivalentes)	0	5
Mutation score	**73,71%**	**98,56%**

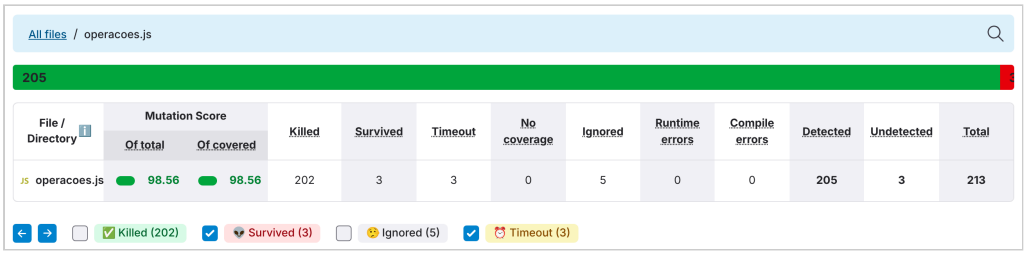


Figura 5 – Resultado final do Stryker.

O score subiu de 73,71% para **98,56%**, acima da meta de 98%. Mesmo sem ignorar os equivalentes do fatorial, ficaria em 96,71%. Todos os mutantes sem cobertura passaram a ser executados e mortos.

5. CONCLUSÃO

O que mais me chamou atenção foi que a suíte original parecia ótima pelo relatório de cobertura e mesmo assim deixava passar um quarto dos bugs simulados. Cobertura mede o que rodou e o teste de mutação mede o que foi realmente verificado. O Stryker apontou com precisão onde os testes eram fracos: fronteiras, caminhos que retornam `false`, mensagens de erro e valores que mascaram a operação. Quase todos os testes novos são pequenos, o que mostra que o problema não era quantidade de testes, e sim a escolha das entradas e das asserções. A ferramenta tem custo, já que os equivalentes exigem análise manual e a execução fica mais lenta em projetos grandes. Mesmo assim, ela mostra a qualidade dos testes de forma muito mais honesta do que a cobertura sozinha. Os mutantes do fatorial ainda revelaram código redundante.